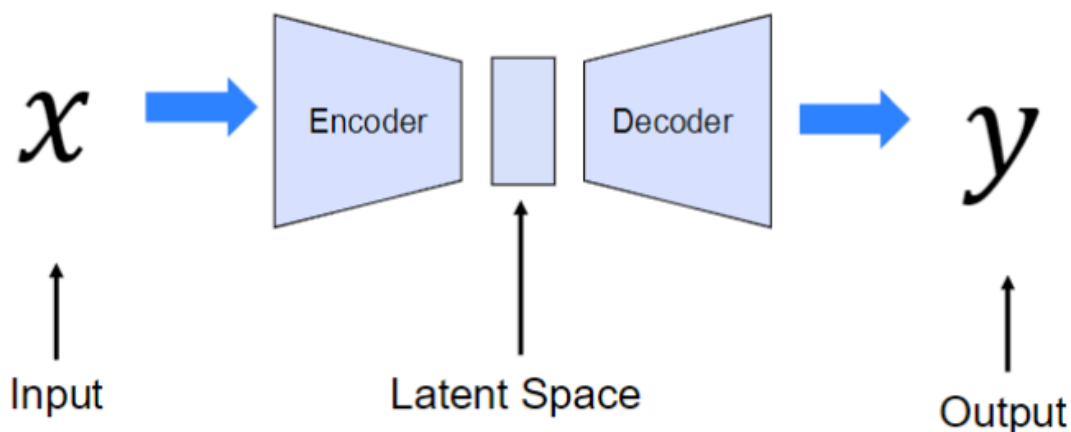


Session 08 - Encoder, Prompting, Reinforcement learning and Applications

Encoder-Decoder Networks

An **encoder-decoder** network is a type of neural architecture commonly used when:

- The **input** and **output** are both sequences or structured data (e.g. images, time series, sentences).
- We need to **transform** one representation into another (e.g. image $\rightarrow$ caption, text $\rightarrow$ translation, image $\rightarrow$ image).



It is composed of two parts:

- **Encoder:** Compresses the input into a compact **latent representation** (z)
 - it gradually **reduces** dimensionality
- **Decoder:** Reconstructs or generates output from z
 - gradually **increases** dimensionality
- The latent space z connects both

Latent Space

- **Latent** means "hidden" or "not directly observed", so the values inside there are not human-interpretable (like pixels or words), they represent **abstract features** learned by the model (e.g., for an image of a cat, it might represent things like "fur texture", "pose", "ears", etc., without explicitly naming them)

 **Latent Space is not a network itself, it is an architecture!**

So it might look like this for example:

```
[Input layer x]
→ [Convolution / Dense / LSTM layers] ← Encoder
→ [Latent vector z]                    ← Latent space
→ [Deconvolution / Dense / LSTM layers] ← Decoder
→ [Output layer y]
```

Autoencoder

An **autoencoder** is a special type of encoder-decoder network where the **output** is intended to **reconstruct the input** so $y \approx x$

- It learns to compress and decompress data efficiently
- **Loss**: focuses on **reconstruction error**

Use Cases:

- Dimensionality reduction
 - Denoising
 - Anomaly detection
 - Pretraining
-

Variational Autoencoder (VAE)

A **VAE** is a **probabilistic extension** of the autoencoder, that instead of mapping an input to a fixed point, it maps it to a **distribution** in latent space.

This allows the model to generate new samples by sampling from the **latent distribution**.

- **Reconstruction loss**: just like a normal autoencoder
→ measures how well the input is reconstructed

Use Cases:

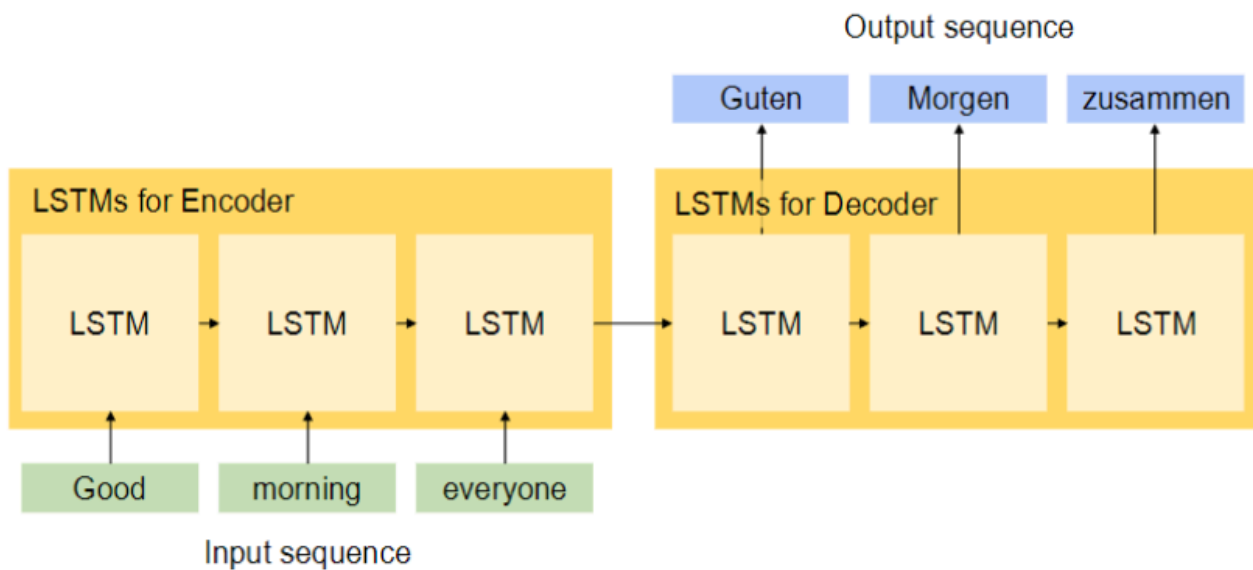
- Generative models (generate realistic data)
- Data interpolation / sampling
- Learning smooth latent spaces

Autoencoder vs Variational Autoencoder vs Encoder-Decoder

- **Autoencoder**: Is like a photocopier that learns to clean up blurry or noisy images: Great for things like **denoising old photos** or **compressing large files** without losing too much info.
 - **Variational Autoencoder (VAE)**: it can learn the *idea* of a thing: So it can then **generate new, slightly different things**. Great for generating **game levels**.
 - **Encoder-Decoder**: it is like a translator that reads one language and writes in another: It is used when **input and output are different things**, like **translating text**, **generating captions for images**, or **turning audio into text**.
-

Sequence-to-Sequence Learning: Language to Language Translation

This can be used when both the input and output are sequences. Like **translating a sentence** from English to German:



1. Encoder:

- Takes the **input sentence** word-by-word, e.g.
Good → morning → everyone
- Each word is passed **one at a time** into the LSTM.
- The LSTM "remembers" the meaning and structure as it goes storing it in its hidden state.
- At the end, the encoder compresses the **entire sentence's meaning** into a **fixed-size vector** (called a *context vector*).

1. Decoder:

- Starts with the **context vector** from the encoder.
- Then it **generates one word at a time** in the target language, e.g.
Guten → Morgen → zusammen
- Each predicted word is used as input for predicting the next one.

Zero Shot Prompting



The model only gives the answer based on natural language that describes the task. Without successive improvements.

One-Shot Prompting

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← example
3	cheese =>	← prompt

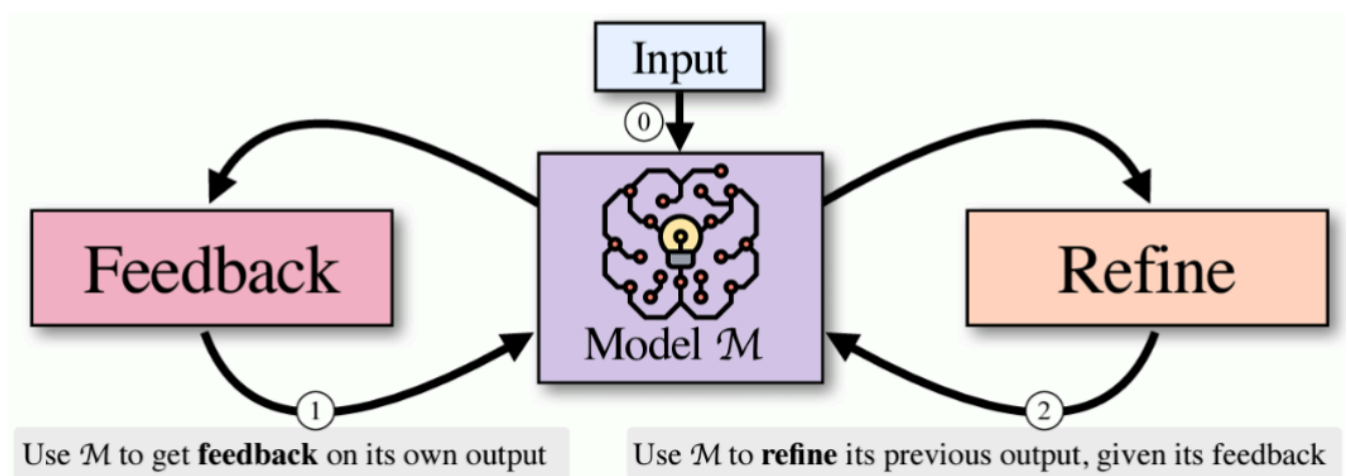
In addition to the task description, the model also receives an example of the task. Without successive improvements.

Few Shot Prompting

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← examples
3	peppermint => menthe poivrée	
4	plush girafe => girafe peluche	
5	cheese =>	← prompt

In addition to the task description, the model is given several examples of the task. Without successive improvements.

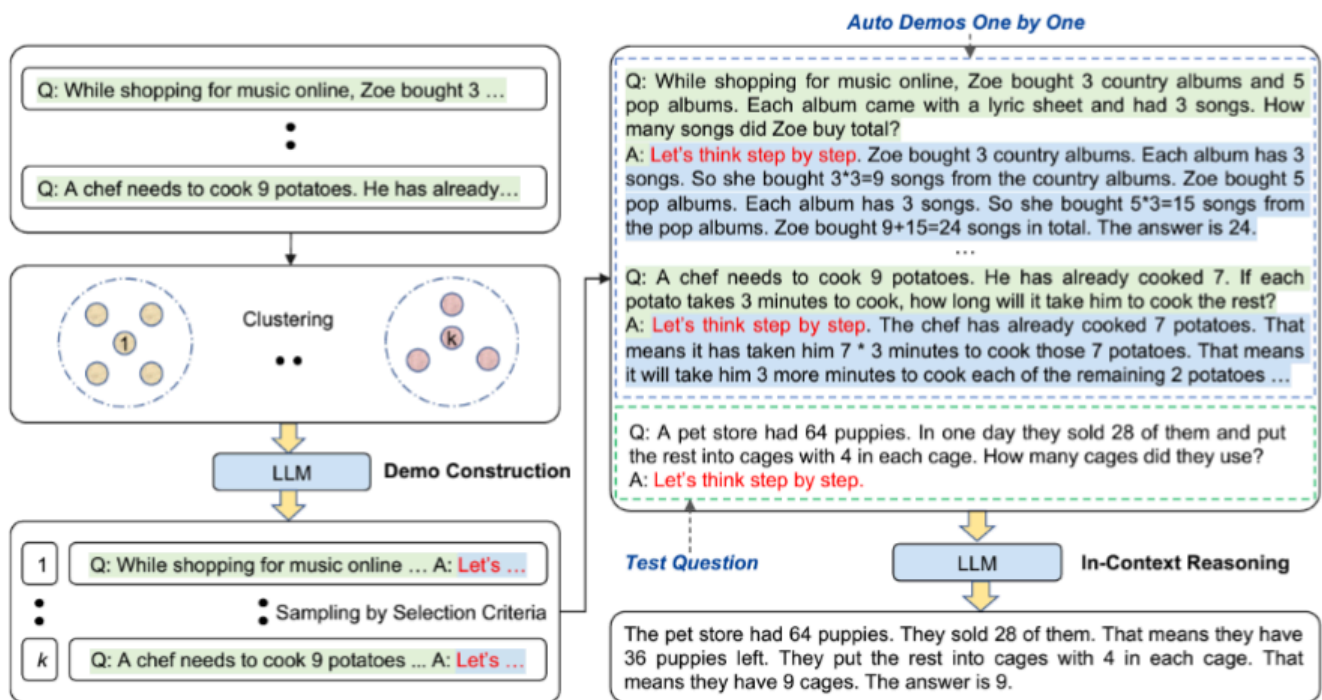
Self-Refine and Chain-of-Thought Prompting



- **Self-Refine:** AI iteratively improves outputs based on feedback.
- **Chain-of-Thought (CoT):** AI explains reasoning steps, improving logical consistency.

Automatic Chain-of-Thought Prompting

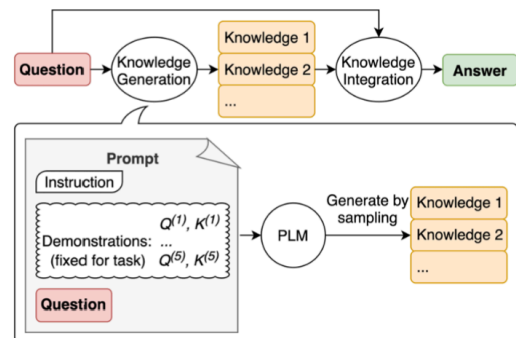
This is a technique that enables LLMs to automatically generate step-by-step reasoning processes, similar to human thinking, to solve complex problems without requiring manual human effort for demonstrations.



Prompting with Knowledge

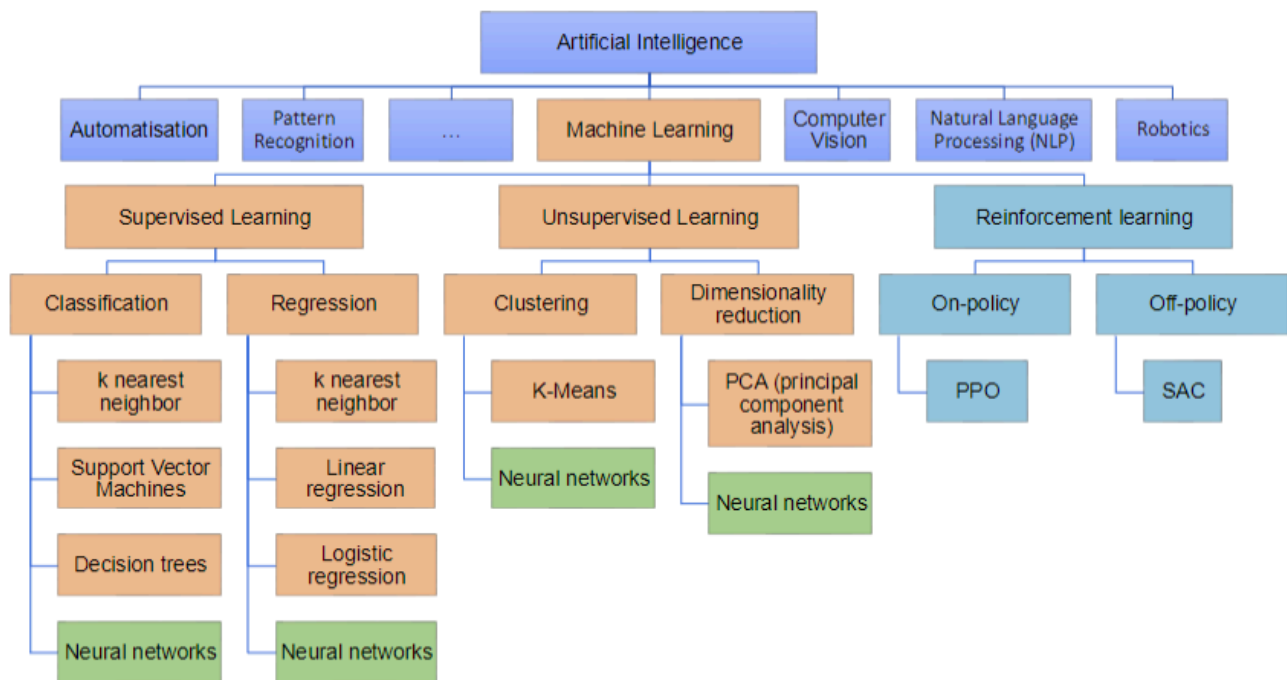


Knowledge Generation Prompting



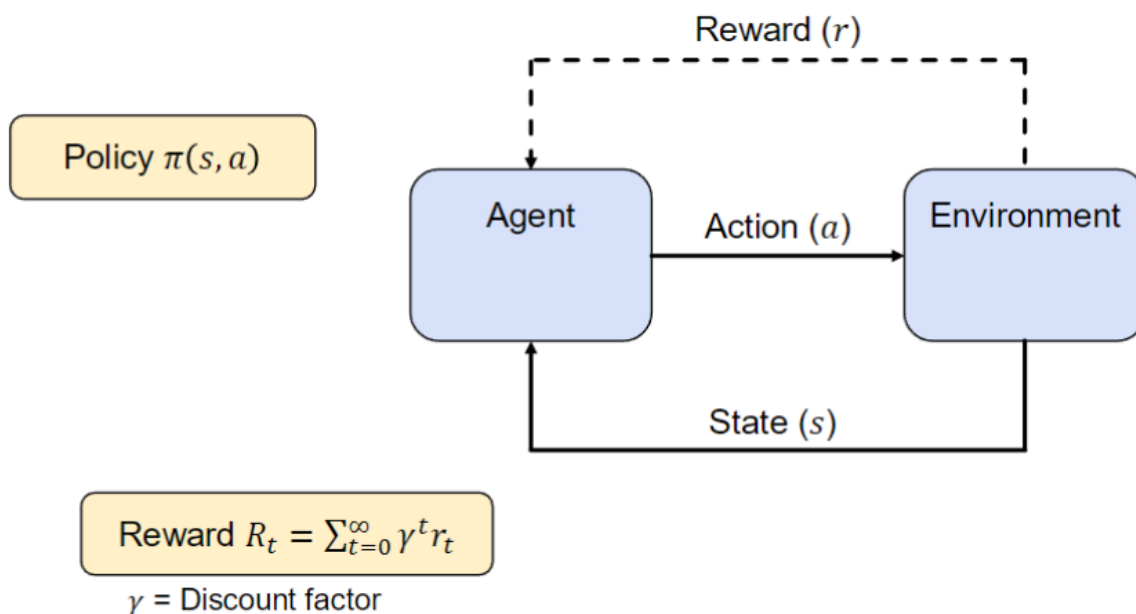
The knowledge contained in the prompt does not require any information to answer the question, but it “guides” the answer in the right direction. This Knowledge can also be generated automatically, though it might not align perfectly with the asked question.

ML Paradigms



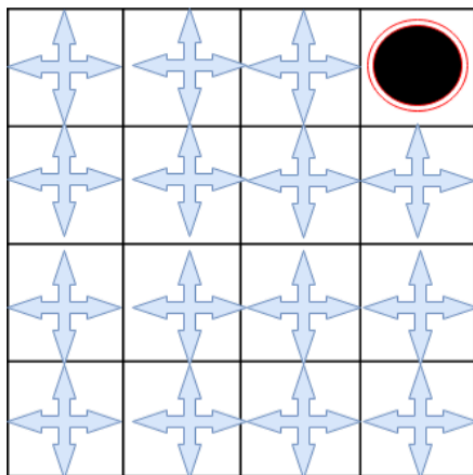
Reinforcement Learning

This involves an agent learning to make sequential decisions by interacting with an environment, receiving rewards for desirable actions and penalties for undesirable ones, with the goal of maximizing cumulative reward over time.

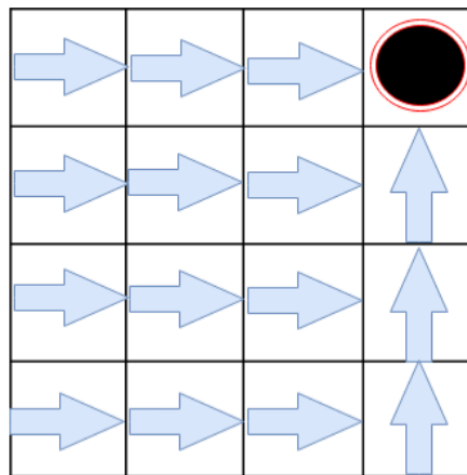


For such an agent there are 2 policies:

1. Target Policy $\pi(a, s)$: It is the policy that an agent is trying to learn i.e agent is learning value function for this policy.
2. Behavior Policy $b(a, s)$: It is the policy that is being used by an agent for action select i.e agent follows this policy to interact with the environment.



Behavior Policy



Target Policy

Example of Behavior and Target Policy, image made via <https://app.diagrams.net/>

On-Policy learning:

- On-Policy learning algorithms are the algorithms that evaluate and improve the same policy which is being used to select actions. That means we will try to evaluate and improve the same policy that the agent is already using for action selection. In short, Target Policy == Behavior Policy. Some examples of On-Policy algorithms are Policy Iteration, Value Iteration, Monte Carlo for On-Policy, Sarsa, etc.

Off-Policy Learning:

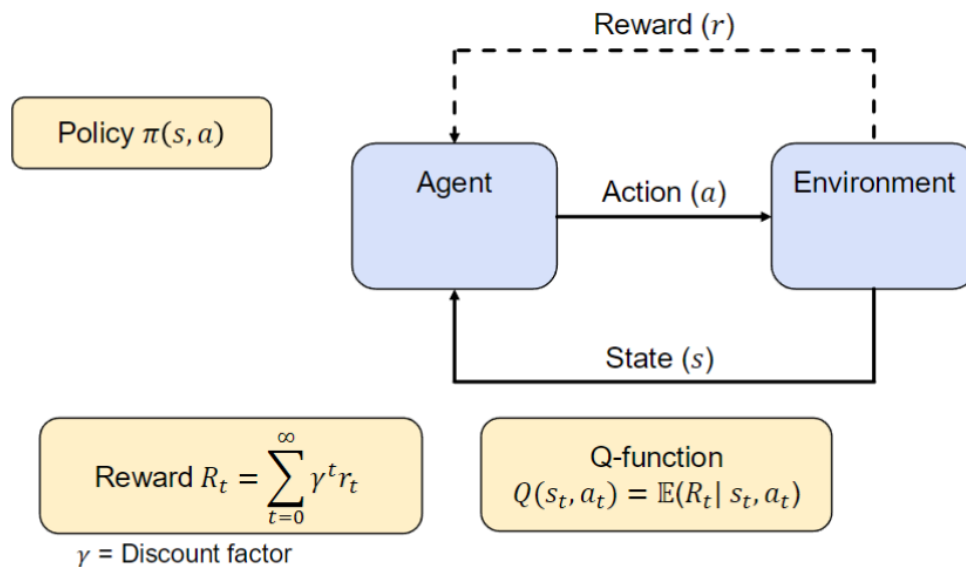
- Off-Policy learning algorithms evaluate and improve a policy that is different from Policy that is used for action selection. In short, Target Policy != Behavior Policy. Some examples of Off-Policy learning algorithms are Q learning, expected sarsa(can act in both ways), etc.

🔗 Behavior policy must cover the target policy i.e $\pi(a|s) > 0$ where $b(a|s) > 0$.

[Article on On-Policy v/s Off-Policy Learning](#)

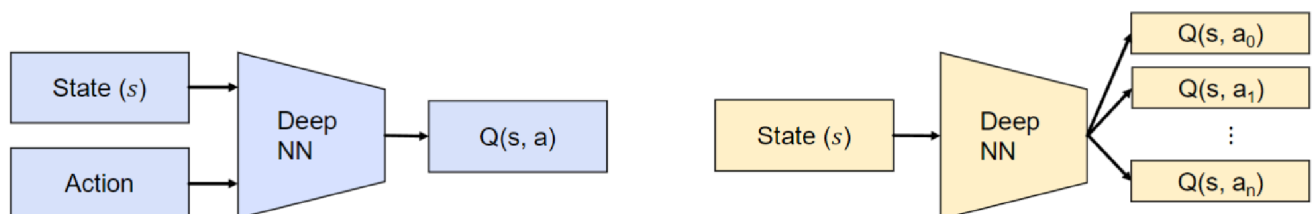
(Deep) Q Learning

Q-learning is a reinforcement learning algorithm that trains an agent to assign values to its possible actions based on its current state, without requiring a model of the environment (it is *model-free*). It can handle problems with stochastic transitions and rewards without requiring adaptations.



"Q" refers to the function that the algorithm computes: the expected reward (that is, the *quality* of an action taken) in a given state. The goal is to approximate the Q-function

Deep Q Networks (DQN)



A neural network taking a state s as input and outputting the **Q-value function** (left light blue network) or all the vectors of Q-values, one for each possible action a (right yellow network).

The right one just illustrates how the Q-value function looks inside.

The $Q(s, a_0), Q(s, a_1), \dots, Q(s, a_n)$, shows each single $Q(s, a_i)$ with it's estimated maximum discounted future reward achievable by taking action a_i in state s and then following the optimal policy thereafter.

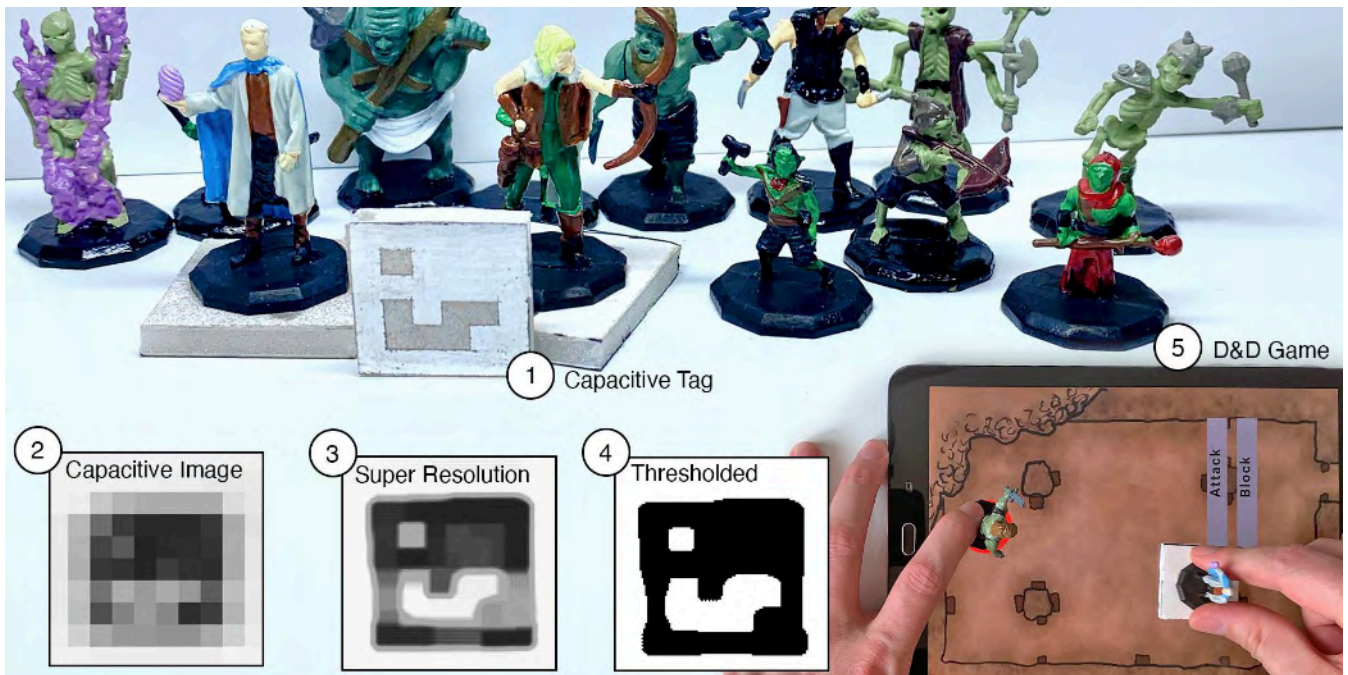
Selecting the action a_k with the highest Q-value in this output vector (e.g., using an argmax operation), as this action is considered the best according to the learned policy.

🔥 The Q value function

With the given state-action value function $Q(s, a)$ we have a direct mapping from state s to expected returns for all available actions. Making it easy to choose the best 'path' to go.

Multi element recognition of 3D-Printed Tangibles (Session 1)

Looking again at the example from session 1 we now want to look at how we can detect more than one element placed on a screen.



Recap:

We were using a CNN that takes a capacitive image (e.g. $27 \times 27 \times 1$) and outputs:

- **Size** (3 classes): 1×3 softmax vector
- **Shape** (10 classes): 1×10 softmax vector
- **Rotation** (1×1 regression): a single scalar angle or orientation

This can detect **a single object** per frame.

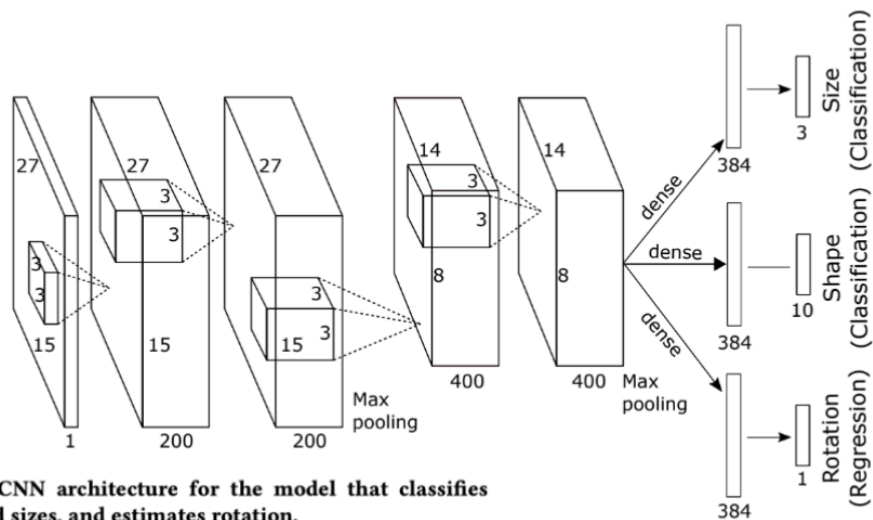


Figure 7: CNN architecture for the model that classifies shapes and sizes, and estimates rotation.

Martin Schmitz, Florian Müller, Max Mühlhäuser, Jan Riemann, and Huy Viet Viet Le. 2021. Itsy-Bits: Fabrication and Recognition of 3D-Printed Tangibles with Small Footprints on Capacitive Touchscreens. Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. Association for Computing Machinery, New York, NY, USA, Article 419, 1–12. DOI: <https://doi.org/10.1145/3411764.3445502>

The old architecture as reference! The new architecture would still mostly be the same as in session 1 only the output layers are different.

Goal: Detect Multiple Objects in the Same Frame

If we were to place up to **up to 10 objects at once** on the touchscreen, each having:

- a unique position
- a marker shape

- a size class
 - and a rotation value.
-

Step-by-Step Architecture Modifications

1. Input Representation

- No change here if we just keep the input size as one whole capacitive image (27×27). The CNN should now **process the entire image** and **internally learn to find multiple objects**.
- If we want to capture more we would need to increase the input area (e.g. 64×64).

2. CNN Body

- The **feature extractor** (convolutions, pooling) does not need to change as we still want to look at the same features.

3. Output Representation

- a). **Size Classification (Was 1×3 → Now 10×3)**
 - Instead of one 3-class softmax, we now want **10 separate softmax predictions**, each predicting the size class of one detected object.
 - Final shape: **(10, 3)**
 - Each row = 1 object
 - Each row is a softmax over 3 size classes.
- b) **Shape Classification (Was 1×10 → Now 10×10)**
 - Similar to above. Predicting the shape class (e.g. tower, wall, animal, dog, human...) for up to 10 objects.
 - Final shape: **(10, 10)**
 - Each row = object
 - Each row = softmax over 10 possible shapes.
- c) **Rotation Regression (Was 1×1 → Now 10×1 or 10×4)**
 - We want one **angle (or orientation vector)** per detected object.
 - We have two choices here:
 - **10×1**: if we output just a single rotation angle (in radians or degrees).
 - **10×4**: if we include **position + rotation** in a single regression head. For example: $[x, y, \cos(\theta), \sin(\theta)]$ per object. This helps disambiguate rotation and is smooth to regress.
 - The shape the Prof. had in mind: **(10 × 4)**
 - Reason: easier learning of angles via $\cos(\theta), \sin(\theta)$ and includes the position.

4. Detection & Object Matching

- Now the model outputs **10 sets** of features. But what if fewer than 10 objects are on screen?
 - It might be a good idea to add a mechanism to **filter valid detections**. Common approaches:
 - Adding **confidence score** for each of the 10 predicted object slots (like YOLO).
 - Adding a “null” class to softmax outputs.
 - Training with padding: unused slots get special targets like shape=[0,...,0], rotation=0.

5. Training: Loss Functions

We can also use a **per-object** losses and sum/average over all 10 object slots:

- **Shape loss**: categorical cross-entropy (10×10)
- **Size loss**: categorical cross-entropy (10×3)
- **Rotation loss**: MSE or cosine similarity (10×4)

We can apply **masking** during training if only some rows are valid (e.g. 4 objects in view → only first 4 rows are meaningful).

The modified network might then look like this (based on the most simple adjustment of just changing the output size):

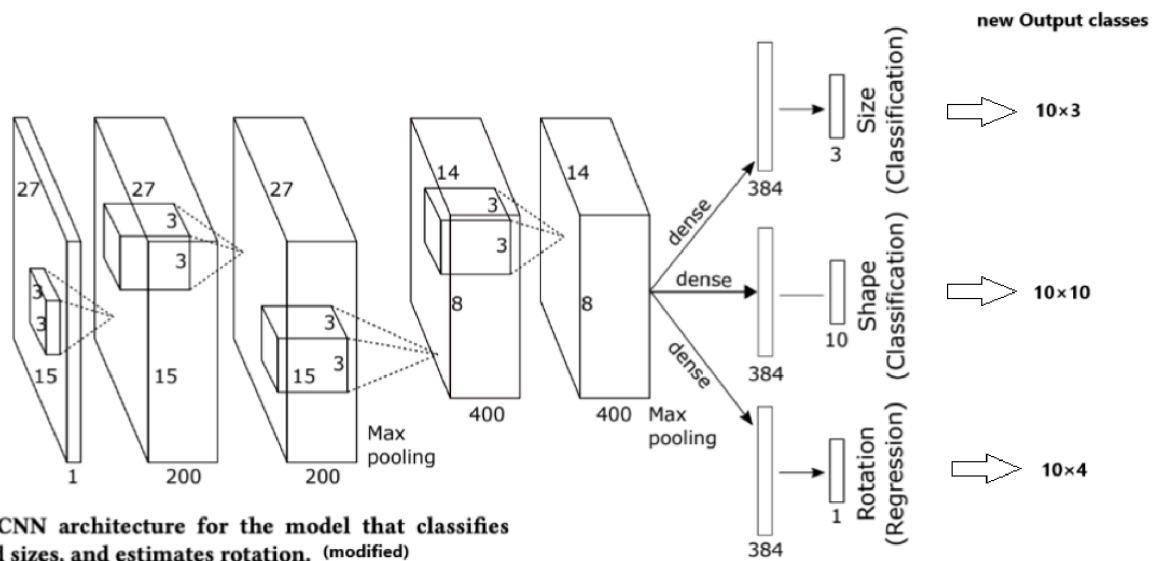


Figure 7: CNN architecture for the model that classifies shapes and sizes, and estimates rotation. (modified)

For the classification softmax could be used.